

Implementation of Rabin Karp Algorithm in C++

Last Updated : 23 Jul, 2025



The Rabin-Karp Algorithm is a string-searching algorithm that efficiently finds a pattern within a text using hashing. It is particularly useful for finding multiple patterns in the same text or for searching in streams of data. In this article, we will learn how to implement the Rabin-Karp Algorithm in C++.

Example:

Input:

`T[] = "THIS IS A TEST TEXT", P[] = "TEST"`

Output:

Pattern found at index 10

What is the Rabin-Karp Algorithm for Pattern Searching?

The Rabin-Karp Algorithm uses hashing to find patterns in strings. It calculates a hash value for the pattern and compares it to the hash values of substrings in the text. If the hash values match, it performs a character-by-character comparison to confirm the match. This algorithm is efficient for average and best-case scenarios, making it suitable for real-world applications.

Algorithm

- *Initialize the following variables:*
 - *m: length of the pattern.*
 - *n: length of the text.*
 - *d: number of characters in the input alphabet (256 in this case).*
 - *q: a prime number used for hash calculation.*
 - *p: hash value for the pattern.*
 - *t: hash value for the current window of text.*
 - *h: helper value for rolling hash calculation, equal to $(d^{(m-1)}) \% q$.*
- *Calculate initial hash values:*

- Compute the hash value p for the pattern.
- Compute the hash value t for the first window of the text (first m characters).
- Slide the pattern over the text: For each position i in the text (from 0 to $n-m$):
 - If the hash values of the pattern and current text window match ($p == t$):
 - Perform a character-by-character comparison of the pattern with the current text window.
 - If all characters match, report a pattern occurrence at index i .
- If $i < n-m$ (not at the end of the text):
 - Calculate the hash value for the next text window using the rolling hash technique:
 - Remove the contribution of the first character of the current window.
 - Add the contribution of the next character after the current window.
- Ensure the new hash value is positive by adding q if necessary.
- Repeat step 3 until the end of the text is reached.

Below is the Illustration of above algorithm:

- Given Text = 315265 and Pattern = 26
- We choose $b = 11$
- $P \bmod b = 26 \bmod 11 = 4$

3	1	5	2	6	5	$31 \bmod 11 = 9$ not equal to 4
3	1	5	2	6	5	$15 \bmod 11 = 4$ equal to 4 -> spurious hit
3	1	5	2	6	5	$52 \bmod 11 = 8$ not equal to 4
3	1	5	2	6	5	$26 \bmod 11 = 4$ equal to 4 -> an exact match!!
3	1	5	2	6	5	$65 \bmod 11 = 10$ not equal to 4

As we can see, when a match is found, further testing is done to ensure that a match has indeed been found.

Rabin Karp Algorithm



Hash Calculation

The hash function used in this algorithm is:

$$H = (d * H + \text{ASCII value of next character}) \% q$$

Here,

- H** is the current hash value.
- d** is the number of characters in the input alphabet.
- q** is a prime number.

Rolling Hash

The algorithm uses a rolling hash technique to efficiently calculate hash values for subsequent windows:

- Remove the contribution of the first character: $t = t - \text{text}[i] * h$
- Shift the remaining hash value: $t = t * d$
- Add the contribution of the new last character: $t = t + \text{text}[i+m]$
- Take the modulus with q : $t = t \% q$

This allows for **O(1)** time complexity when sliding the window, making the overall average-case time complexity $O(n+m)$, where n is the length of the text and m is the length of the pattern.

C++ Program to Implement Rabin Karp Algorithm

The following program demonstrates the implementation of Rabin Karp Algorithm:

```
// C++ Program for Implementation of Rabin-Karp Algorithm
```

```
#include <iostream>
#include <string>
using namespace std;
// Number of characters in the input alphabet
#define d 256

void rabinKarp(string pattern, string text, int q)
{
    int m = pattern.length();
    int n = text.length();
    int i, j;
    // Hash value for pattern
    int p = 0;
    // Hash value for text
    int t = 0;
    int h = 1;

    // The value of h would be "pow(d, m-1)%q"
    for (i = 0; i < m - 1; i++)
        h = (h * d) % q;

    // Calculate the hash value of pattern and first window
    // of text
    for (i = 0; i < m; i++) {
        p = (d * p + pattern[i]) % q;
        t = (d * t + text[i]) % q;
    }

    // Slide the pattern over text one by one
    for (i = 0; i <= n - m; i++) {
        // Check the hash values of current window of text
        // and pattern
```

```

    if (p == t) {
        // Check for characters one by one
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }
        if (j == m)
            cout << "Pattern found at index " << i
                << endl;
    }

    // Calculate hash value for next window of text
    if (i < n - m) {
        t = (d * (t - text[i] * h) + text[i + m]) % q;
        if (t < 0)
            t = (t + q);
    }
}
}

// Driver Code
int main()
{
    string text = "GEEKSFORGEEKS";
    string pattern = "GEEKS";
    int q = 101;

    rabinKarp(pattern, text, q);
    return 0;
}

```

Output

```

Pattern found at index 0
Pattern found at index 8

```

Time Complexity:

- The average and best-case running time of the Rabin-Karp algorithm is $O(n+m)$, but its worst-case time is $O(nm)$.
- The worst case of the Rabin-Karp algorithm occurs when all characters of pattern and text are the same as the hash values of all the substrings of $T[]$

match with the hash value of P[].

Auxiliary Space: $O(1)$

Limitations of Rabin-Karp Algorithm

When the hash value of the pattern matches with the hash value of a window of the text but the window is not the actual pattern then it is called a spurious hit.

Spurious hit increases the time complexity of the algorithm. In order to minimize spurious hit, we use good [hash function](#). It greatly reduces the spurious hit.

Related Posts

- [Searching for Patterns | Set 1 \(Naive Pattern Searching\)](#)
- [Searching for Patterns | Set 2 \(KMP Algorithm\)](#)

Suggested Quiz

🔄 5 Questions

What is the primary purpose of the Rabin-Karp algorithm?

- ☐ A To efficiently find a pattern within a text using hashing
- ☐ B To perform character-by-character comparison of the pattern with the text
- ☐ C To calculate the hash value for the entire text
- ☐ D To slide the pattern over the text one character at a time

1/5

< Previous

Next >

💬 Comment

 **striver...** [+ Follow](#)

👍 0 

Article Tags :

C++

CPP-DSA

Explore

C++ Basics 

Core Concepts 

OOP in C++ 

Standard Template Library(STL) 

Practice & Problems 

Do Not Sell or Share My Personal Information